

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 20%

Code of Conduct:

I G. Manish 192210591 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Manish

RUBRICS FOR CALCULATION:

9/12
W

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent.

Analyze the issue and explain how preprocessing improves feature detection performance.

Code:

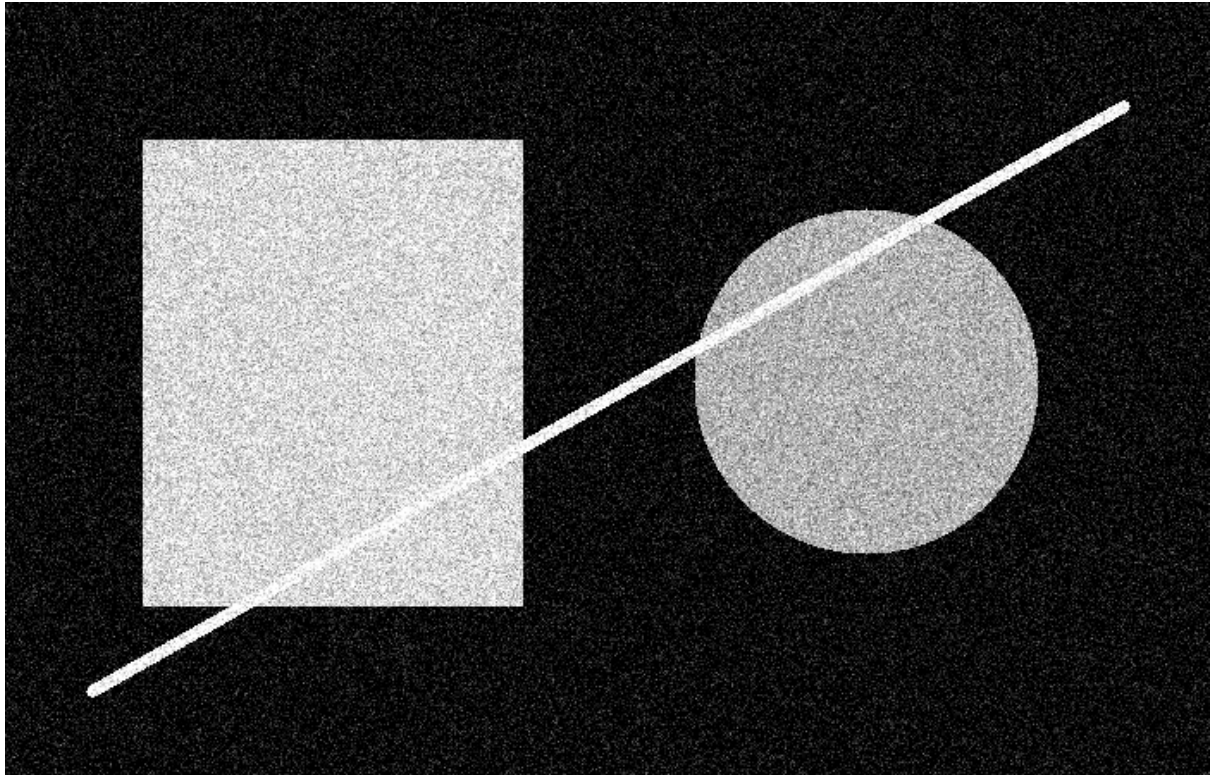
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)
denoised = cv2.GaussianBlur(img, (5,5), 0)
enhanced = cv2.equalizeHist(denoised)

orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(img, None)
kp2, des2 = orb.detectAndCompute(enhanced, None)

print("Keypoints before preprocessing:", len(kp1))
print("Keypoints after preprocessing :", len(kp2))

plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(cv2.drawKeypoints(img, kp1, None), cmap="gray")
plt.title("Before preprocessing"); plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(cv2.drawKeypoints(enhanced, kp2, None), cmap="gray")
plt.title("After preprocessing"); plt.axis("off")
plt.show()
```



2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching.

Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Code:

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
img = cv2.imread("input.jpg", 0)
```

```
edges = cv2.Canny(img, 100, 200)
```

```
corners = cv2.goodFeaturesToTrack(img, 100, 0.01, 10)
```

```
out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
```

```
if corners is not None:
```

```
    for c in corners.astype(int):
```

```
        x,y = c.ravel()
```

```
        cv2.circle(out, (x,y), 4, (0,255,0), -1)
```

```
plt.figure(figsize=(10,4))
```

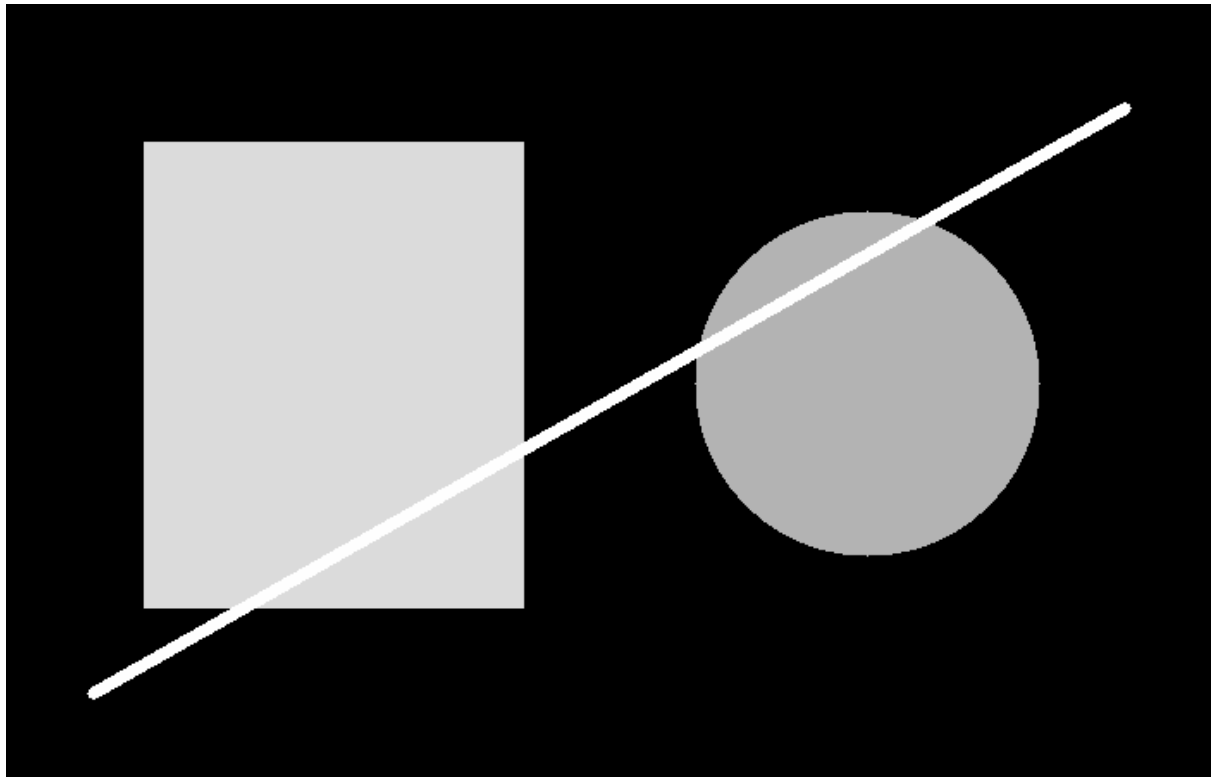
```
plt.subplot(1,2,1); plt.imshow(edges,cmap="gray"); plt.title("Canny Edges");
```

```
plt.axis("off")
```

```
plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(out,cv2.COLOR_BGR2RGB));
```

```
plt.title("Corner Points"); plt.axis("off")
```

```
plt.show()
```



3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.

Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Code:

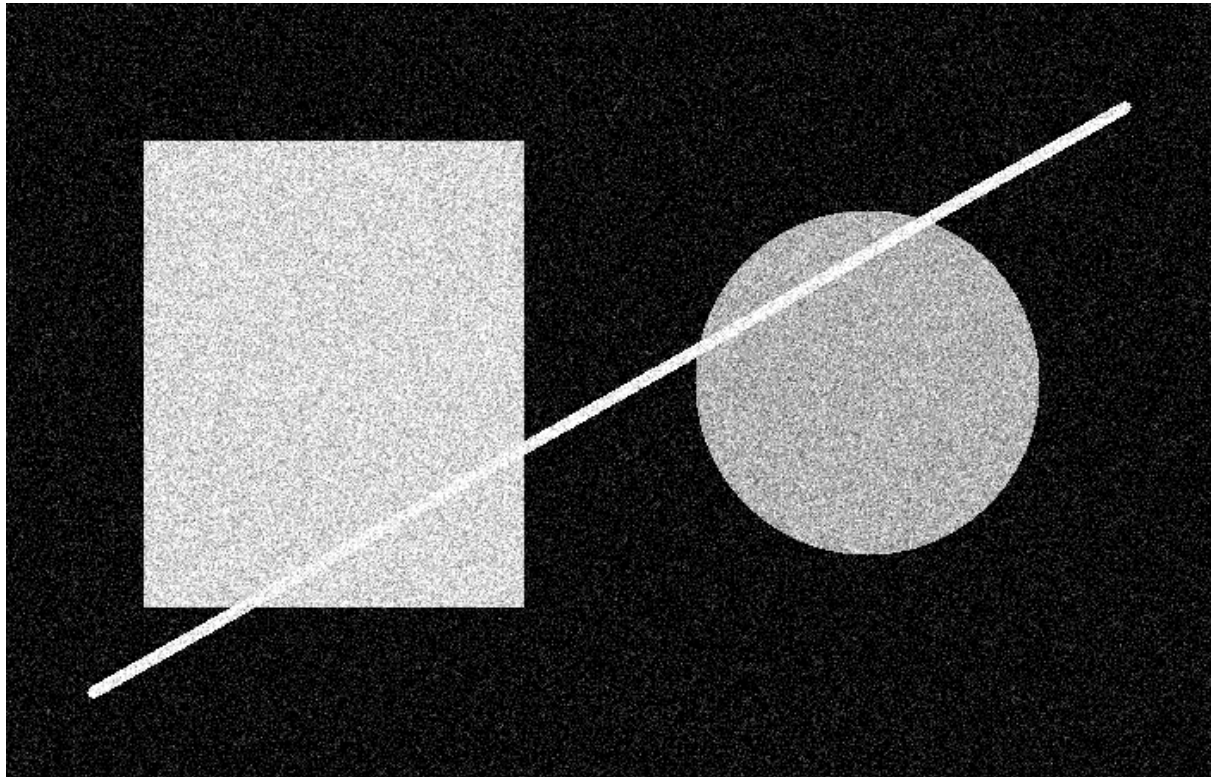
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)
blur = cv2.GaussianBlur(img, (5,5), 0)
edges = cv2.Canny(blur, 50, 150)

lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=60,
                        minLineLength=50, maxLineGap=10)

out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
if lines is not None:
    for line in lines:
        x1,y1,x2,y2 = line[0]
        cv2.line(out,(x1,y1),(x2,y2),(0,255,0),2)
```

```
plt.imshow(cv2.cvtColor(out,cv2.COLOR_BGR2RGB))
plt.title("Hough Line Detection after Denoising")
plt.axis("off"); plt.show()
```



4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively.

Analyze why scale invariant methods perform better in this scenario.

Code:

```
import cv2
import numpy as np

# Create a sample reference image
img1 = np.zeros((500, 700), dtype=np.uint8)

cv2.rectangle(img1, (150, 100), (500, 400), 255, 5)
cv2.circle(img1, (325, 250), 80, 255, 5)
cv2.line(img1, (150, 100), (500, 400), 255, 4)
cv2.line(img1, (500, 100), (150, 400), 255, 4)

# Create a transformed version:
# rotation + scaling
```

```

center = (350, 250)
matrix = cv2.getRotationMatrix2D(center, 30, 1.3)

img2 = cv2.warpAffine(
    img1,
    matrix,
    (700, 500)
)

# SIFT
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

print("Reference image keypoints:", len(kp1))
print("Transformed image keypoints:", len(kp2))

# Match descriptors
bf = cv2.BFMatcher()

matches = bf.knnMatch(des1, des2, k=2)

# Lowe's ratio test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good scale/rotation invariant matches:",
      len(good_matches))

# Draw matches
result = cv2.drawMatches(
    img1,
    kp1,
    img2,
    kp2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

```

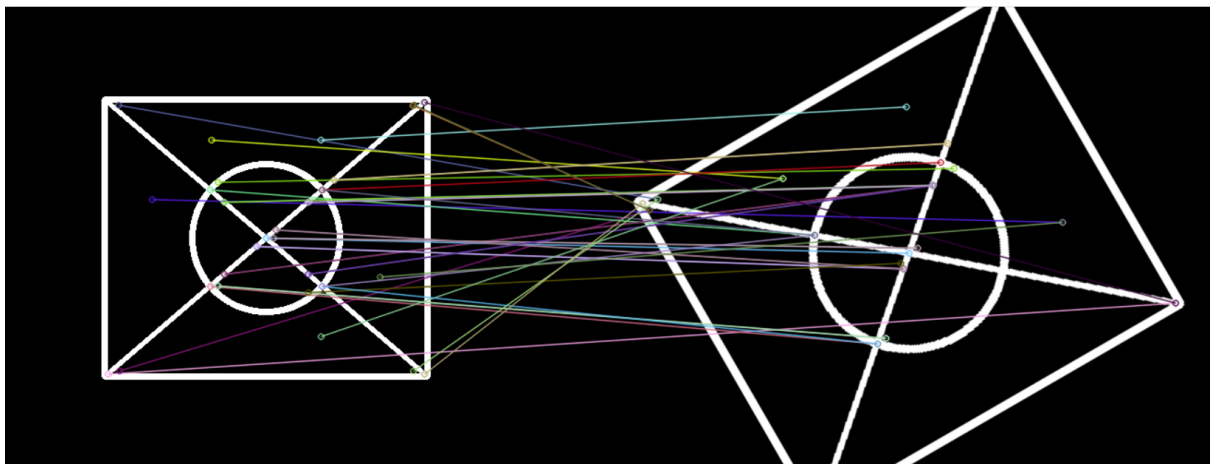
```

cv2.imwrite(
    "04_scale_rotation_matching_result.png",
    result
)

cv2.imshow(
    "SIFT Scale and Rotation Invariant Matching",
    result
)

cv2.waitKey(0)
cv2.destroyAllWindows()

```



5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy.

Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Code:

```

import cv2
import numpy as np
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

X, y = load_digits(return_X_y=True)

pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X)

print("Original dimensions :", X.shape[1])
print("Reduced dimensions :", X_reduced.shape[1])
print("Variance retained :", pca.explained_variance_ratio_.sum())

```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.grid()
plt.show()
```

```
1 import cv2
2 import numpy as np
3 from sklearn.decomposition import PCA
4 from sklearn.datasets import load_digits
5 import matplotlib.pyplot as plt
6
7 X, y = load_digits(return_X_y=True)
8
9 pca = PCA(n_components=0.95)
10 X_reduced = pca.fit_transform(X)
11
12 print("Original dimensions :", X.shape[1])
13 print("Reduced dimensions  :", X_reduced.shape[1])
14 print("Variance retained   :", pca.explained_variance_ratio_.sum())
15
16 plt.plot(np.cumsum(pca.explained_variance_ratio_))
17 plt.xlabel("Number of components")
18 plt.ylabel("Cumulative explained variance")
19 plt.grid()
20 plt.show()
```

6. Preprocessing Impact on Edge Detection

A system performs edge detection on raw images containing noise and uneven illumination, resulting in fragmented edges.

Analyze the problem and explain how preprocessing improves edge detection accuracy and continuity.

Code:

```
import cv2
import matplotlib.pyplot as plt

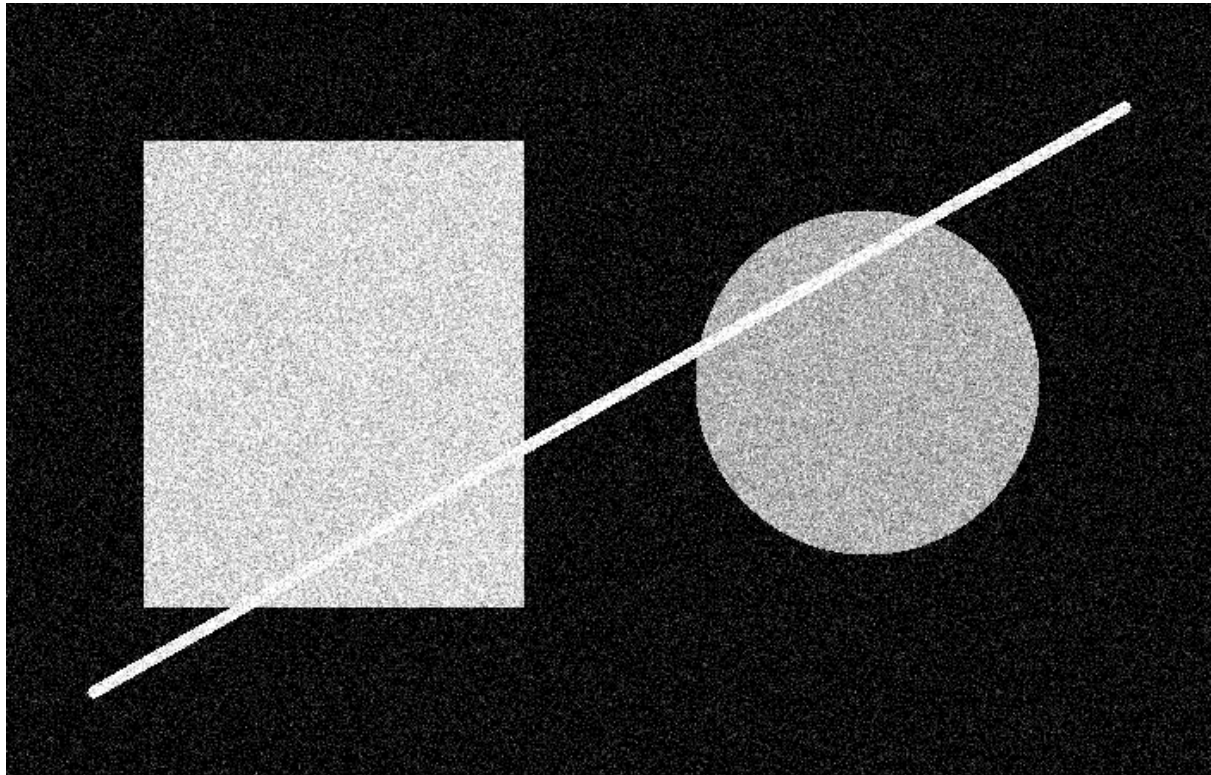
img = cv2.imread("input.jpg", 0)
denoised = cv2.GaussianBlur(img, (5,5), 0)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
enhanced = clahe.apply(denoised)

raw_edges = cv2.Canny(img, 50, 150)
clean_edges = cv2.Canny(enhanced, 50, 150)

plt.figure(figsize=(10,4))
```



```
plt.subplot(1,2,1); plt.imshow(raw_edges,cmap="gray"); plt.title("Raw Image Edges");
plt.axis("off")
plt.subplot(1,2,2); plt.imshow(clean_edges,cmap="gray"); plt.title("Preprocessed
Edges"); plt.axis("off")
plt.show()
```



7. Corner Detection vs Edge Detection in Tracking

A tracking system uses edge detection but fails to track moving objects accurately across frames.

Analyze why edges are insufficient and justify the use of corner detection for reliable tracking.

Code:

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
old_gray = None
```

```
old_pts = None
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret: break
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    if old_pts is None:
```

```

old_pts = cv2.goodFeaturesToTrack(gray, 100, 0.01, 10)
old_gray = gray.copy()
else:
    new_pts, status, _ = cv2.calcOpticalFlowPyrLK(
        old_gray, gray, old_pts, None
    )
    if new_pts is not None:
        for p,n,s in zip(old_pts, new_pts, status):
            if s:
                x,y = n.ravel().astype(int)
                cv2.circle(frame,(x,y),4,(0,255,0),-1)
            old_pts = new_pts[status==1].reshape(-1,1,2)
        old_gray = gray.copy()

cv2.imshow("Corner Tracking", frame)
if cv2.waitKey(1) & 0xFF == 27: break

cap.release(); cv2.destroyAllWindows()

```

8. Hough Transform for Shape Detection

A system uses Hough Transform to detect circular shapes but fails in complex backgrounds.

Analyze the limitations and explain how preprocessing or parameter tuning can

Code:

improve detection. import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)

blur = cv2.medianBlur(img, 5)

circles = cv2.HoughCircles(

blur, cv2.HOUGH_GRADIENT, dp=1.2, minDist=30,

param1=100, param2=30, minRadius=10, maxRadius=150

)

out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

if circles is not None:

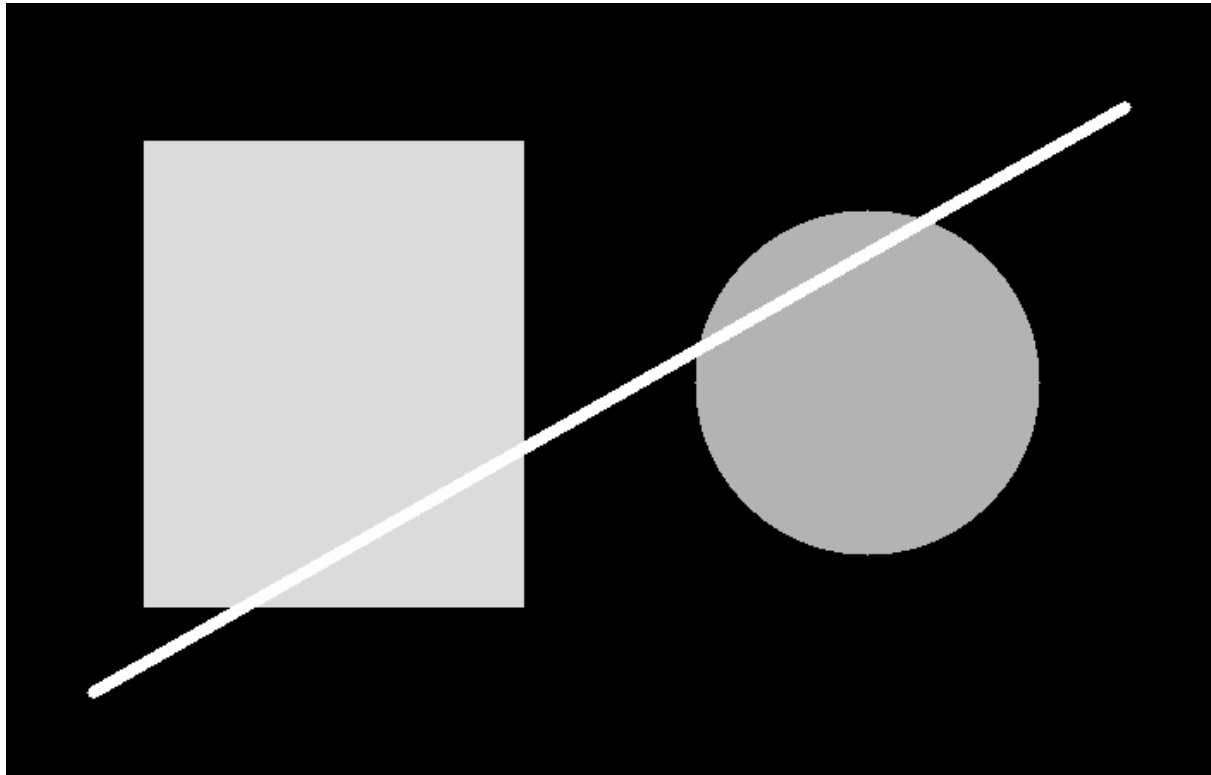
circles = np.uint16(np.around(circles))

for x,y,r in circles[0,:]:

cv2.circle(out,(x,y),r,(0,255,0),2)

cv2.circle(out,(x,y),2,(0,0,255),3)

```
plt.imshow(cv2.cvtColor(out,cv2.COLOR_BGR2RGB))
plt.title("Hough Circle Detection")
plt.axis("off"); plt.show()
```



9. Feature Matching under Illumination Changes

Two images of the same scene are captured under different lighting conditions, causing feature matching errors.

Analyze why matching fails and explain how preprocessing or robust descriptors improve performance.

Code:

```
import cv2
import numpy as np

# -----
# Create a sample scene
# -----

img1 = np.zeros((500, 700), dtype=np.uint8)

# Objects in the scene
cv2.rectangle(img1, (120, 100), (580, 400), 180, -1)
cv2.rectangle(img1, (170, 150), (530, 350), 80, -1)
cv2.circle(img1, (350, 250), 90, 220, -1)
```

```
# Add some lines and shapes for feature points
cv2.line(img1, (120, 100), (580, 400), 255, 4)
cv2.line(img1, (580, 100), (120, 400), 255, 4)
cv2.rectangle(img1, (220, 180), (300, 260), 200, -1)
```

```
# -----
# Create second image with different illumination
# -----
```

```
# Make image darker
img2 = cv2.convertScaleAbs(
    img1,
    alpha=0.55,
    beta=20
)
```

```
# Add slight brightness variation
illumination = np.zeros_like(img2)
cv2.circle(illumination, (550, 100), 250, 50, -1)
```

```
img2 = cv2.add(img2, illumination)
```

```
# -----
# Apply CLAHE
# -----
```

```
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
```

```
img1_enhanced = clahe.apply(img1)
img2_enhanced = clahe.apply(img2)
```

```
# -----
# SIFT Feature Detection
# -----
```

```
sift = cv2.SIFT_create()
```

```
kp1, des1 = sift.detectAndCompute(
    img1_enhanced,
    None
)
```

```

kp2, des2 = sift.detectAndCompute(
    img2_enhanced,
    None
)

print("Keypoints in Image 1:", len(kp1))
print("Keypoints in Image 2:", len(kp2))

# -----
# Feature Matching
# -----

matcher = cv2.BFMatcher()

matches = matcher.knnMatch(
    des1,
    des2,
    k=2
)

# Lowe's Ratio Test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print(
    "Good matches after illumination normalization:",
    len(good_matches)
)

# -----
# Draw Matching Result
# -----

result = cv2.drawMatches(
    img1_enhanced,
    kp1,
    img2_enhanced,
    kp2,
    good_matches,
    None,

```

```

    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

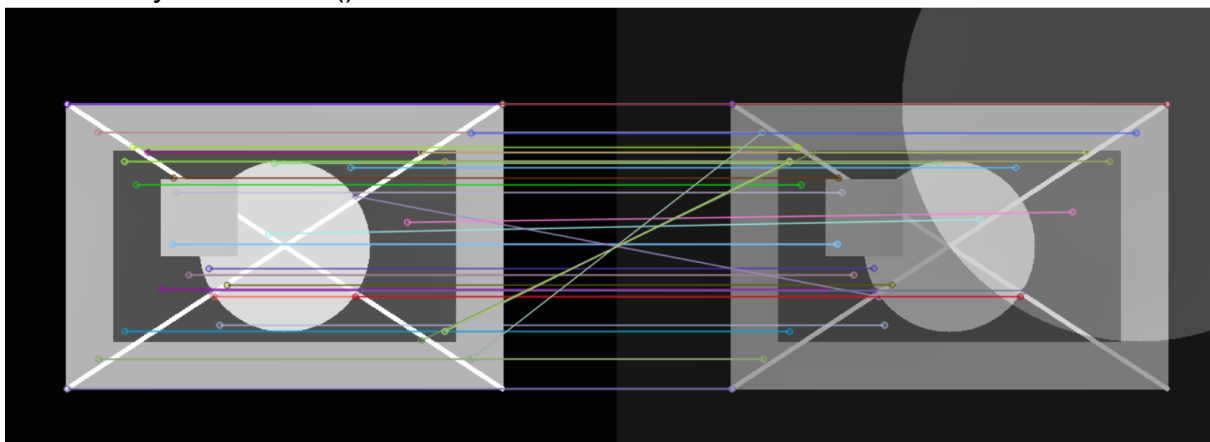
# Save result
cv2.imwrite(
    "09_illumination_matching_result.png",
    result
)

print(
    "Result saved as: 09_illumination_matching_result.png"
)

# Display result
cv2.imshow(
    "Feature Matching under Illumination Changes",
    result
)

cv2.waitKey(0)
cv2.destroyAllWindows()

```



10. PCA in Face Recognition

A face recognition system processes high-dimensional image data, resulting in slow performance.

Analyze how PCA helps in reducing dimensionality while preserving important features.

11. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyze the problem and explain how preprocessing enhances edge detection results.

Code:

```

import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
enhanced = clahe.apply(img)

edges_before = cv2.Canny(img, 50, 100)
edges_after = cv2.Canny(enhanced, 50, 100)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(edges_before,cmap="gray"); plt.title("Before Contrast Enhancement"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(edges_after,cmap="gray"); plt.title("After CLAHE"); plt.axis("off")
plt.show()

```



12. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints.

Analyze the issue and suggest methods to improve feature selection.

13. Feature Matching with Noise

Feature matching between two images fails due to noise in one image.

Analyze the impact of noise and explain how preprocessing improves matching accuracy.

14. PCA and Information Loss

A system applies PCA aggressively and loses important feature details.

Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

15. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection.

Analyze the issue and explain how appropriate feature descriptors improve matching performance

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible	Logical solutions	Basic solutions with weak	Incorrect or no solution

		solutions with strong justification	with adequate justification	justification	provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand